

初版需求描述

背景

雷总让我帮他写软著，我想着最近一直听说Harness架构大家觉得还比较靠谱，但是我从来没有实践过，所以想着借此机会来用Harness这种设计架构来落地一个小系统，这个系统就是专门帮我写软著的一个小系统，

起的名字叫【墨衡 DocForge】

墨 = 写作、文档生成。

衡 = 审核、校准、打分、把关。

产品定义

墨衡 DocForge 是一个面向软件著作权文档生成的多智能体文档生产系统。它根据外部参考软著样例、自家公司产品资料、需求文档，自动完成产品理解、文档规划、初稿生成、审计评分、多轮修订和最终导出。

范围和负向约束

MVP 要做的：

1. 读取参考软著样例（支持多格式，docx,pdf, md ,txt
2. 读取目标产品介绍文档
3. 自动生成一版软著文档草稿
4. 对软著草稿进行最多 3 轮“审计-修订”循环，并输出最终版或风险版
5. MVP 只做本地轻量前端界面，用于文件上传（参考文档和本地资料，本文设计注意事项2中已经明确规定）、产品类型选择、方案确认、结果下载。先不要做复杂生产级 Web UI。MVP 用 **Streamlit**，不上 React/Vue。

负向约束：

1. 先不要接代码仓库，先暂时不通过URL读取网页。
2. 先不要做复杂权限。
3. 先不要接公司知识库。

4. 先不要做多格式高级排版。
5. 不做多租户
6. 不做在线协作编辑

技术栈

代码块

- 1 开发工具：
- 2 - Cursor
- 3
- 4 运行环境：
- 5 - macOS
- 6 - Python 3.11 或 3.12
- 7
- 8 核心框架：
- 9 - LangGraph
- 10
- 11 模型层：（放弃本地模型，就用线上模型，默认Qwen）
- 12 - QwenProvider
- 13 - DeepSeekProvider
- 14
- 15
- 16 RAG / 检索：
- 17 - Qdrant
- 18
- 19
- 20 文档解析：
- 21 - docx: python-docx
- 22 - pdf: pypdf / pymupdf
- 23 - markdown/txt/html: 常规文本解析
- 24 - 图片: 视觉模型解析 + OCR 兜底
- 25
- 26
- 27 数据存储：
- 28 - 原始文件: 本地文件夹
- 29 - 结构化任务状态: JSON / SQLite
- 30 - 向量证据库: Qdrant
- 31 - 最终产物: 《软件著作权文档》.docx
- 32
- 33 轻量前端：
- 34 - Streamlit
- 35 用途：
- 36 - 上传资料

```
37 - 选择产品类型
38 - 展示软件类型诊断结果
39 - 确认模板策略
40 - 用户在和AI的对话中，可以下载软著文档：《软件著作权文档》.docx
41
42 导出工具：
43 - python-docx
```

核心原则

代码块

```
1  核心原则与系统规则
2  本节规则必须做成系统规则，不允许只靠提示词临场发挥。
3
4  一、资料用途隔离原则
5  规则 1：参考软著只能学写法，不能提供产品事实
6  外部参考软著只能用于学习：
7  1. 目录结构
8  2. 章节顺序
9  3. 标题写法
10 4. 描述方式
11 5. 配图方式
12 6. 文档语气
13 7. 软著常见表达
14 不得复用参考软著中的：
15 1. 产品名称
16 2. 产品功能
17 3. 业务描述
18 4. 模块说明
19 5. 技术架构
20 6. 截图内容
21 系统规则：
22 参考软著资料必须标记为：
23 corpus_type = reference_style
24 allowed_usage = style_only
25 规则 2：自有产品资料才是事实来源
26 我方产品资料才允许作为产品事实来源，包括：
27 1. 产品介绍文档
28 2. PRD
29 3. HLD
30 4. 详细设计文档
31 5. 产品截图
32 6. 系统原型截图
```

33 7. 用户补充说明
34 系统规则：
35 自有产品资料必须标记为：
36 `corpus_type = product_evidence`
37 `allowed_usage = factual_evidence`
38 规则 3：产品截图属于强证据
39 产品截图是软著文档中非常重要的证据来源。
40 如果截图中能明确看到页面、按钮、菜单、表格、流程、功能入口，则该截图可作为强证据。
41 系统规则：
42 产品截图默认标记为 `product_evidence`。
43 产品截图可用于支撑页面功能、操作步骤、截图配图和功能说明。
44 规则 4：用户补充说明属于弱证据
45 用户补充说明可以作为事实来源，但默认属于弱证据。
46 系统规则：
47 用户补充说明可以进入 `product_evidence`。
48 但关键功能、当前版本状态、上线范围等内容，需要用户确认后才能写成确定事实。
49 规则 5：产品 URL 是 Phase 2 预留能力，MVP 禁用
50 MVP 阶段不接入、不解析、不截图、不入库 URL。
51 系统规则：
52 MVP 阶段，`MaterialIngestionAgent` 不接收 URL 输入。
53 MVP 阶段，不实现 `URLReader`。
54 MVP 阶段，不实现 `SiteCapture`。
55 MVP 阶段，不做网页自动截图。
56 MVP 阶段，不处理登录态页面。
57 MVP 阶段，不做站点爬取。
58 即使 State 中保留 `product_url` 相关字段，也必须默认置空，不进入 workflow。
59
60 二、资料接入规则
61 规则 6：MVP 只接收四类资料
62 MVP 阶段，`MaterialIngestionAgent` 只接收：
63 1. 参考软著资料
64 2. 自有产品资料
65 3. 产品截图
66 4. 用户补充说明
67 不接收：
68 1. 产品 URL
69 2. 代码仓库
70 3. 公司知识库
71 4. 在线协作文档
72 5. 外部网页爬取结果
73 规则 7：所有资料必须统一转成 `Evidence Unit`
74 MVP 中，PRD、HLD、产品介绍文档、产品截图、用户补充说明都必须统一转成 `Evidence Unit`。
75 系统真正需要的不是原始文件，而是可检索、可引用、可审计的证据单元。
76 `Evidence Unit` 至少包含：
77 1. `evidence_id`
78 2. `source_id`
79 3. `source_type`

80 4. *file_type*
81 5. *corpus_type*
82 6. *allowed_usage*
83 7. *evidence_type*
84 8. *evidence_strength*
85 9. *summary*
86 10. *extracted_facts*
87 11. *tags*
88 12. *source_location*
89 13. *confidence*
90 14. *is_confirmed*
91 15. *needs_human_confirmation*
92 代码类资料属于 Phase 2, 不进入 MVP。
93
94 三、写作与审计规则
95 规则 8: WriterAgent 写作必须能追溯事实来源
96 WriterAgent 写正文时, 所有产品事实必须能追溯到 EvidenceItem。
97 系统规则:
98 WriterAgent 查询产品事实时, 必须使用过滤条件:
99 *corpus_type* = *product_evidence*
100 *allowed_usage* = *factual_evidence*
101 WriterAgent 不得使用 *reference_style* 资料生成产品事实。
102 规则 9: 证据不足的功能不得强写
103 如果某功能没有来自自有资料、产品截图或用户确认的证据, 则不得写成当前版本已实现功能。
104 系统规则:
105 证据不足的功能只能标记为“需确认功能”。
106 不得把推测功能写成确定功能。
107 不得为了让文档完整而自行补造功能。
108 规则 10: AI 能力必须区分已实现和规划中
109 如果资料中出现模型、训练、推理、评测等关键词, 可以增加“AI 平台增强包”。
110 但如果这些内容只出现在规划章节、未来规划、路线图或待建设说明中, 不得写成当前版本已实现功能。
111 系统规则:
112 规划中功能可以进入 *risk_notes* 或 *uncertain_features*。
113 当前版本功能必须有 *product_evidence* 支撑。
114
115 规则 11: AuditAgent 必须检查参考资料污染
116 AuditAgent 必须检查是否出现 *reference_style* 污染 *product_evidence* 的问题。
117 重点检查:
118 1. 是否把参考软著中的功能写成我方功能
119 2. 是否把参考软著中的产品名称写入正文
120 3. 是否复用了参考软著中的业务描述
121 4. 是否把 *reference_style* EvidenceItem 当作 *factual_evidence* 使用
122 如果出现上述问题, 判定为 Blocker。
123 规则 12: 审计循环必须有限
124 审计修订采用“评分阈值 + 最大轮次”。
125 系统规则:

126 max_revision_round = 3
127 不得无限循环修订。
128 如果 v3 仍未通过 DraftQualityGate, 则导出:
129 《软件著作权文档_风险版》.docx
130
131 四、模板诊断规则
132 规则 13: 有 Web 页面截图, 默认优先 SaaS 用户手册型
133 如果资料中存在 3 张以上 Web 页面截图, 并且包含登录页、首页、功能页面, 则默认推荐:
134 Web/SaaS 用户操作手册型
135 该规则只影响模板推荐, 不代表可以自行补造页面功能。
136 规则 14: 数据对象强, 就自动加数据平台增强包
137 如果资料中高频出现以下关键词:
138 数据集、数据资产、样本、标注、质检、导入、导出、质量报告、数据处理
139 则增加:
140 数据平台增强包
141 规则 15: AI 能力强, 就自动加 AI 平台增强包
142 如果资料中高频出现以下关键词:
143 模型、训练、推理、评测、Embedding、LoRA、微调、数据集训练、模型资产
144 则增加:
145 AI 平台增强包
146 但必须同时区分:
147 1. 当前版本已实现功能
148 2. 规划中功能
149 3. 证据不足功能
150
151 五、计划冻结规则
152 规则 16: 用户确认后生成 FrozenDocPlan
153 用户确认模板策略后, OrchestratorAgent 必须生成 FrozenDocPlan。
154 FrozenDocPlan 是后续文档生成的合同。
155 它控制:
156 1. 软件身份
157 2. 软件类型
158 3. 模板策略
159 4. 一级目录
160 5. 功能范围
161 6. 证据策略
162 7. 截图策略
163 8. 写作规则
164 9. Quality Gate 规则
165 规则 17: 用户确认后一级目录冻结
166 用户确认模板后, 后续 Agent 不得擅自更改一级目录。
167 系统规则:
168 OutlineAgent 只能基于 FrozenDocPlan 生成详细目录。
169 WriterAgent 只能基于 FrozenDocPlan 写正文。
170 AuditAgent 必须检查草稿是否违反 FrozenDocPlan。
171 如需调整一级目录, 必须重新进入用户确认节点。
172

173 六、存储与工程边界规则
174 规则 18: State 只保存任务状态和证据指针
175 State 只保存当前任务状态、文件路径、source_id、evidence_id、draft_id、
audit_report_id、Qdrant collection 名和结构化摘要。
176 State 不保存：
177 1. 完整原始文档
178 2. 完整截图
179 3. 完整 DOCX
180 4. 大段原始资料全文
181 规则 19: Qdrant 只保存可检索证据，不保存工作流状态
182 Qdrant 只保存 Evidence Unit 的向量和 payload。
183 Qdrant 不保存：
184 1. LangGraph 工作流状态
185 2. 完整原始文档
186 3. 完整截图
187 4. 完整 DOCX
188 5. state.json
189 规则 20: 模型调用必须通过 Provider Adapter
190 所有模型调用必须通过 Provider Adapter。
191 系统规则：
192 Agent 不得直接绑定某一家模型厂商 SDK。
193 MVP 默认支持：
194 1. QwenProvider
195 2. DeepSeekProvider
196 后续可扩展其他模型厂商。

Agent团队设计

墨衡 DocForge Agent Team如下：

后续如果引用，就叫：

代码块

1 AT-02: 软著生成 Agent 团队 v2.0

1. OrchestratorAgent：总指挥

这是必须有的，不然流程会乱。

它负责：

代码块

- 1 1. 控制整个 workflow 状态
- 2 2. 调度各个 Agent
- 3 3. 保存中间结果
- 4 4. 判断是否进入用户确认
- 5 5. 判断是否进入 Quality Gate
- 6 6. 控制 v1/v2/v3 重写次数
- 7 7. 生成 FrozenDocPlan
- 8 8. 防止 WriterAgent 越权修改目录
- 9 9. 触发最终导出

它不是写作 Agent，而是状态机 + 总编排者。

核心状态：

代码块

- 1 ProjectState
- 2 - project_id
- 3 - software_name
- 4 - version
- 5 - owner
- 6 - source_materials
- 7 - reference_style_result
- 8 - product_understanding_result
- 9 - evidence_map
- 10 - diagnosis_result
- 11 - template_strategy
- 12 - frozen_doc_plan
- 13 - outline
- 14 - draft_versions
- 15 - audit_reports
- 16 - current_quality_status
- 17 - export_result

没有 OrchestratorAgent，你们后面很难管理：

代码块

- 1 用户确认过什么？
- 2 哪版目录是最终的？
- 3 v2 是根据哪份审计报告改的？
- 4 截图是从哪里来的？

2. IntakeAgent：项目接入 Agent

负责创建软著项目。

采集最小必要信息：

代码块

1. 软件名称
2. 软件版本
3. 著作权人
4. 软件用途简述
5. 当前是否已有产品页面
6. 是否有参考软著
7. 是否有自有产品资料

MVP 不要一开始问太多。

先让任务跑起来。

3. MaterialIngestionAgent：资料接入与分类 Agent

负责把资料分成几类：

代码块

1. 参考软著资料
2. 自有产品资料
3. 产品截图
4. 用户补充说明

核心规则：

代码块

- 1 参考软著 $\neq$ 事实来源
- 2 自有产品资料 = 事实来源
- 3 产品截图 = 强证据
- 4 产品 URL = 可解析事实来源（Phase 2 可扩展事实来源，MVP这个版本不做）

这个边界必须写进系统规则。

否则最容易把别人公司的软著内容写进自己文档。

4. ProductSourceAgent: 产品来源读取 Agent

这是你第 3 点提到的新能力。

它负责从用户自己的产品资料中获取信息，包括：

代码块

1. 读取上传文档
2. 读取上传截图
3. 提取产品资料中的功能、页面、模块、流程

这个 Agent 可以拆成三个子能力：

代码块

- 1 DocumentReader：读文档
- 2 ScreenshotReader：读截图
- 3 ~~URLReader / SiteCapture：读网页和自动截图（这个去掉了，暂时不做）~~
- 4
- 5 MVP 禁用能力：
- 6 URLReader / SiteCapture 不实现。
- 7 MVP 不读取网页 URL，不做网页自动截图，不处理登录态页面，不做站点爬取。
- 8
- 9 Phase 2 可扩展：
- 10 如果后续要支持 URL，再新增 URLReader / SiteCapture，并重新设计网页解析、截图权限、登录态、反爬处理和证据可信度规则。

5. ReferenceStyleAgent: 参考软著风格分析 Agent

只分析参考文档的：

代码块

1. 目录结构
2. 章节粒度
3. 功能描述方式

- 4 4. 截图插入方式
- 5 5. 标题命名风格
- 6 6. 操作步骤写法
- 7 7. 常见固定章节

禁止提取为事实的内容：

代码块

- 1 1. 对方产品名称
- 2 2. 对方功能模块
- 3 3. 对方业务描述
- 4 4. 对方截图内容
- 5 5. 对方技术架构

输出：

代码块

- 1 ReferenceStyleProfile
- 2 - common_chapter_structure
- 3 - writing_style
- 4 - screenshot_usage_pattern
- 5 - operation_step_pattern
- 6 - reusable_outline_pattern
- 7 - prohibited_content_warning

6. ProductUnderstandingAgent：产品理解 Agent

负责理解自有产品。

输出：

代码块

- 1 ProductUnderstandingResult
- 2 - product_positioning
- 3 - target_users
- 4 - core_modules
- 5 - core_workflows
- 6 - business_objects
- 7 - page_list
- 8 - feature_list
- 9 - technical_keywords

```
10 - industry_keywords
11 - uncertain_features
```

它不负责定目录。

它只负责“理解产品是什么”。

7. EvidenceExtractorAgent：证据抽取 Agent

负责构建证据地图。

输出：

```
代码块

1 EvidenceMap
2 - function_name
3 - source_type
4 - source_file_or_url
5 - source_location
6 - screenshot_id
7 - evidence_strength
8 - confidence
9 - notes
```

证据强度建议分级：

证据类型	强度
产品真实页面截图	强
自有 PRD/HLD/产品说明	强
产品 URL 页面内容	强
用户口头补充	中
参考软著	只能参考风格，不能作为事实证据

8. SoftwareDiagnosisAgent：软件类型诊断 Agent

负责判断：

```
代码块
```

- 1 1. 主类型
- 2 2. 核心业务对象
- 3 3. 增强标签
- 4 4. 推荐文档写法

输出示例：

代码块

```
1  DiagnosisResult
2  - primary_type: Web/SaaS 平台
3  - primary_type_confidence: 0.88
4  - business_objects:
5    - 数据资产
6    - 数据集
7  - enhancement_tags:
8    - AI 平台
9    - 工业软件
10   - 汽车行业
11 - recommended_doc_style: 用户操作手册型软著文档
12 - alternative_doc_styles:
13   - 数据平台增强型
14   - AI 平台增强型
15 - reasons:
16   - 存在多个 Web 页面截图
17   - 产品功能围绕数据导入、数据集管理、质检报告展开
18   - 资料中出现模型训练、评测等 AI 功能
19 - risk_notes:
20   - 模型训练功能需要确认是否已在当前版本实现
```

9. TemplateStrategyAgent：模板策略 Agent

根据诊断结果生成推荐方案。

输出：

代码块

```
1  TemplateStrategy
2  - base_template
3  - enhancement_packs
4  - excluded_templates
5  - recommended_chapters
6  - optional_chapters
```

```
7 - risk_chapters
8 - recommendation_reason
```

例如：

代码块

```
1 base_template: Web/SaaS 用户操作手册型
2 enhancement_packs:
3     - 数据平台增强包
4     - AI 平台轻量增强包
5     - 权限管理增强包
6 excluded_templates:
7     - 后端/API 技术说明型
8     - 嵌入式软件设计说明型
```

10. HumanConfirmGate：用户确认节点

这个不一定是 LLM Agent，也可以是前端交互节点。

展示：

代码块

```
1 根据上传资料，我初步判断：
2
3 主类型：Web/SaaS 平台
4 核心业务对象：数据资产 / 数据集
5 增强标签：AI 平台、工业软件、汽车行业
6
7 因此建议采用：
8 《用户操作手册型软著文档》
9
10 请确认：
11 A. 按推荐类型生成
12 B. 手动修改软件类型
13 C. 查看其它类型模板
```

用户确认后，进入 FrozenDocPlan。

11. OutlineAgent：目录规划 Agent

只允许基于 FrozenDocPlan 生成目录。

不能自己发明新目录。

12. WriterAgent：正文写作 Agent

只允许基于：

代码块

1

FrozenDocPlan

2

DocumentOutline

3

EvidenceMap

4

ReferenceStyleProfile

5

WritingRules

写正文。

它不能：

代码块

1

1. 擅自修改一级目录

2

2. 擅自新增核心功能

3

3. 把未来规划写成已实现

4

4. 复用参考软著的产品内容

13. ScreenshotBindingAgent：截图绑定 Agent

负责：

代码块

1

1. 给章节分配截图

2

2. 生成截图说明

3

3. 判断截图是否缺失

4

4. 判断截图和章节是否匹配

5

5. 生成补充截图建议

14. AuditAgent：审计 Agent

负责：

代码块

- 1 1. 内容真实性检查
- 2 2. 目录一致性检查
- 3 3. 证据支撑检查
- 4 4. 截图匹配检查
- 5 5. 版本一致性检查
- 6 6. 参考污染检查
- 7 7. 软著文档规范性检查

15. ExportAgent：导出 Agent

输出：

代码块

- 1 1. Word 文档，例如：软件著作权文档-260604.docx
- 2 ~~2. 审计报告（这个也先去掉！）~~

代码块

- 1 后续可扩展 Agent：
- 2 - CodeReaderAgent：读取代码仓库，提炼模块、接口、类、数据库表
- 3 - EvidenceCheckerAgent：检查每一段是否有资料依据
- 4 - FormatAgent：控制 docx 格式、标题、目录、页眉页脚
- 5 - LegalRiskAgent：检查是否过度承诺、是否把商业宣传写成技术事实
- 6 - HumanReviewAgent：让人确认关键事实

workflow (Pipeline)

主流程

代码块

- 1 WF-02：软著文档生成 workflow v2.0


```
2
3 OrchestratorAgent 启动任务
4   ↓
5 1. IntakeAgent: 创建软著项目
6   ↓
7 2. MaterialIngestionAgent: 资料接入与分类
8   ↓
9 3. ProductSourceAgent: 产品资料读取
10   └─ 文档读取: PRD / HLD / 产品介绍 / 详细设计文档
11   └─ 图片读取: 用户上传截图
12   ↓
13 4. ReferenceStyleAgent: 参考软著风格分析
14   ↓
15 5. ProductUnderstandingAgent: 自有产品理解
16   ↓
17 6. EvidenceExtractorAgent: 证据抽取与证据地图构建
18   ↓
19 7. SoftwareDiagnosisAgent: 软件类型诊断
20   ↓
21 8. TemplateStrategyAgent: 模板推荐与目录策略生成
22   ↓
23 9. HumanConfirmGate: 用户确认推荐方案
24   └─ A. 按推荐类型生成
25   └─ B. 手动修改软件类型
26   └─ C. 查看其它类型模板
27   ↓
28 10. OrchestratorAgent: 生成并冻结 FrozenDocPlan
29   ↓
30 11. OutlineAgent: 基于 FrozenDocPlan 生成详细目录
31   ↓
32 12. PlanQualityGate: 生成前质量门禁
33   └─ 通过: 进入正文生成
34   └─ 不通过: 反问用户补充关键信息
35   ↓
36 13. WriterAgent: 写 v1 初稿
37   ↓
38 14. ScreenshotBindingAgent: 截图绑定与章节配图
39   ↓
40 15. AuditAgent: 审查 v1
41   ↓
42 16. DraftQualityGate: 是否通过?
43   └─ 是: 进入导出
44   └─ 否: WriterAgent 根据审计意见写 v2
45       ↓
46       AuditAgent 审 v2
47       ↓
48       DraftQualityGate: 是否通过?
```

```
49      └─ 是：进入导出
50      └─ 否：WriterAgent 写 v3
51          ↓
52      AuditAgent 审 v3
53          ↓
54      DraftQualityGate：是否通过？
55          └─ 是：进入导出
56          └─ 否：导出《软件著作权文档_风险版》.docx
57      ↓
58  17. ExportAgent：导出《软件著作权文档》.docx
```

注意事项

1-审计业务设计

1. 审计目标

AuditAgent 的目标不是单纯给软著文档打分，而是判断当前生成的软著文档是否满足以下要求：

- 1. 文档内容是否符合 FrozenDocPlan；
- 2. 文档中的产品事实是否均来自 product_evidence；
- 3. 是否存在 reference_style 资料污染产品事实的问题；
- 4. 是否存在未实现功能被写成已实现功能的问题；
- 5. 软件名称、版本号、目录结构、核心章节是否一致且完整；
- 6. 文档是否符合“用户操作手册型软著文档”的基本写法；
- 7. 当前版本是否可以导出为最终《软件著作权文档》.docx。

2. 问题等级定义

等级	含义	处理方式
Blocker	阻塞级问题，影响事实真实性、流程合法性或核心结构	必须修复；存在任意 Blocker 时不得作为最终版通过
Major	严重质量问题，明显影响文档可用性或可信度	优先修复；数量过多时不得通过
Minor	一般质量问题，不影响核心事实和主要结构	可修复；不阻塞导出
Suggestion	优化建议，不属于质量缺陷	不扣分，不阻塞导出

3. Blocker 问题

出现以下问题，判定为 Blocker：

1. 使用 reference_style 资料作为产品事实来源；
2. 核心功能没有任何 product_evidence 支撑；
3. 软件名称缺失，或全文软件名称严重不一致；
4. 软件版本缺失，或全文软件版本严重不一致；
5. 未生成 FrozenDocPlan 就开始生成正文；
6. 一级目录违反 FrozenDocPlan；
7. 将明确的未来规划功能写成当前版本已实现功能；
8. 文档核心章节缺失，例如软件概述、运行环境、核心功能说明；
9. WriterAgent 擅自新增 FrozenDocPlan 中禁止写入的功能；
10. 最终文档无法形成完整的软著文档结构。

4. Major 问题

出现以下问题，判定为 Major：

1. 核心章节有 product_evidence 支撑，但操作步骤明显不完整；
2. 章节内容和证据弱匹配，存在表达过度或依据不足；
3. FrozenDocPlan 要求必须配图的章节缺少截图，且没有替代文字说明；
4. 术语前后不统一，已经影响用户理解；
5. 运行环境描述过于模糊，无法支撑软著文档基本要求；
6. 部分章节明显偏离“用户操作手册型软著文档”风格；
7. 存在较多宣传式表达，例如“行业领先、全球首创、颠覆性”等；
8. 疑似将未来功能写成已实现功能，但证据不足以判定为 Blocker；
9. 章节内容过薄，无法支撑该章节作为一级或二级章节存在。

5. Minor 问题

出现以下问题，判定为 Minor：

1. 个别句子表达不够规范；
2. 个别标题粒度不均衡；
3. 个别截图说明不够完整；
4. 章节之间详略略有不均；
5. 少量术语不统一，但不影响整体理解；
6. 个别操作步骤表达不够细，但不影响主要功能说明；
7. 个别段落存在重复表达。

6. Suggestion

以下情况属于 Suggestion，不扣分，不阻塞导出：

- 1. 可优化措辞；
- 2. 可增加截图；
- 3. 可补充操作注意事项；
- 4. 可增强章节之间的过渡；
- 5. 可补充更多示例说明；
- 6. 可进一步提升排版观感。

7. 扣分规则

初始分为 100 分。

问题等级	扣分规则	是否自动阻塞通过
Blocker	每个扣 30 分	是
Major	每个扣 8 分	否
Minor	每个扣 3 分	否
Suggestion	不扣分	否

说明：

- 1. Blocker 同时具有“扣分”和“一票否决”作用；
- 2. 即使总分高于 85，只要存在 Blocker，也不得通过；
- 3. Suggestion 只进入优化建议，不参与评分；
- 4. 扣分后的最低分为 0 分，不出现负分。

8. 通过条件

DraftQualityGate 通过必须同时满足：

- 1. score >= 85；
- 2. blocker_count = 0；
- 3. major_count <= 2；
- 4. 必填章节完整；
- 5. 产品事实均来自 product_evidence；
- 6. reference_style 没有被当作产品事实使用；
- 7. 一级目录没有违反 FrozenDocPlan；
- 8. 软件名称和版本号全文一致；
- 9. 没有将明确的未来规划功能写成当前版本已实现功能。

只要任意一项不满足，则本轮 DraftQualityGate 不通过。

9. 审计流程

每次 WriterAgent 生成草稿后，AuditAgent 执行审计：

1. 检查 FrozenDocPlan 遵守情况；
2. 检查产品事实来源；
3. 检查 reference_style 污染风险；
4. 检查软件名称和版本一致性；
5. 检查必填章节完整性；
6. 检查未来规划功能是否被写成当前已实现功能；
7. 检查文档风格是否符合用户操作手册型软著文档；
8. 识别 Blocker、Major、Minor、Suggestion；
9. 按扣分规则计算 score；
10. 给出本轮是否通过的结论。

10. 最大修订轮次

系统最多执行 3 轮 “写作-审计-修订” 循环：

1. WriterAgent 生成 v1；
2. AuditAgent 审计 v1；
3. 如果 v1 通过，则导出最终 DOCX；
4. 如果 v1 不通过，则 WriterAgent 根据审计意见生成 v2；
5. AuditAgent 审计 v2；
6. 如果 v2 通过，则导出最终 DOCX；
7. 如果 v2 不通过，则 WriterAgent 根据审计意见生成 v3；
8. AuditAgent 审计 v3；
9. 如果 v3 通过，则导出最终 DOCX；
10. 如果 v3 仍不通过，则导出风险版 DOCX。

最大修订轮次由参数控制：

代码块

```
1 max_revision_round = 3
```

11. 风险版导出规则

MVP 不单独导出审计报告和风险报告。

如果 v3 仍未通过 DraftQualityGate，系统仍然只导出 1 份 DOCX，但文件名标记为风险版：《软件著作权文档_风险版》.docx

系统内部保留以下调试文件，但不作为用户正式下载产物：

- 1. audit_report.json；
- 2. evidence_map.json；
- 3. state.json；
- 4. draft_v1 / draft_v2 / draft_v3 中间稿。

风险版 DOCX 的生成原则：

- 1. 优先使用当前质量最高的草稿版本；
- 2. 不单独生成审计报告；
- 3. 不把内部审计细节写入正文；
- 4. 文件名标记“风险版”，提醒用户需要人工复核。

关键对象

1、门禁对象：【PlanQualityGate】和【DraftQualityGate】

1. PlanQualityGate：写作前质量门禁

位置：

代码块

- 1 FrozenDocPlan 生成后
- 2 OutlineAgent 生成目录后
- 3 WriterAgent 写正文前

它检查的是：**能不能开始写。**

检查项：

代码块

- 1 PlanQualityGate 检查项 (CheckList) :
- 2
- 3 1. 软件名称是否明确

- 4 2. 软件版本号是否明确
- 5 3. 软件类型是否已确认
- 6 4. 文档模板是否已冻结
- 7 5. 一级目录是否已冻结
- 8 6. 是否区分了参考资料和自有产品资料
- 9 7. 核心功能是否都有证据来源
- 10 8. 核心章节是否有可用截图或替代说明
- 11 9. 是否存在“疑似未来规划功能”
- 12 10. 是否存在必须向用户反问的缺失信息

通过条件：

代码块

- 1 必须满足：
- 2 - FrozenDocPlan 已生成
- 3 - 用户已确认模板策略
- 4 - 核心功能有证据支撑
- 5 - 没有 Blocker 级缺失项

不通过时，不应该直接写。要反问用户。

例如：

代码块

- 1 当前无法直接生成软著文档，请确认以下关键信息：
- 2
- 3 1. 软件版本号是否为 V1.0？
- 4 2. “模型训练”功能是否已经在当前版本上线？
- 5 3. 是否有用户权限管理页面？
- 6 4. 当前系统是否通过浏览器访问？
- 7 5. 是否允许将官网/演示站点截图作为软著配图？

注意：PlanQualityGate 最多问 5~8 个问题。

不能问 30 个。否则用户体验会崩。

2. DraftQualityGate：成稿后质量门禁

位置：

代码块

- 1 WriterAgent 写 v1/v2/v3 后

它检查的是：**这版文档能不能导出。使用 100 分制**

他是需要计算的， $\text{score} = 100 - \text{blocker_count} * 30 - \text{major_count} * 8 - \text{minor_count} * 3$ 。

它的完整逻辑，怎么判断审计是否通过，还是需要再审，请详细查看本文中的【1-审计业务设计】

2、LangGraph的State对象

版本：DocForgeState v2.0，这个2.0版本的State对象专门匹配下面这些：

WF-02：软著文档生成工作流 v2.0

AT-02：软著生成 Agent 团队 v2.0

FDP-01：FrozenDocPlan 数据结构 v1.0

QG-01：双 Quality Gate 机制

实现该State对象前，我们需要注意：

代码块

```

1  ### State 持久化边界
2
3  DocForgeState 是业务状态契约，不等于 LangGraph checkpoint，也不等于 Qdrant 存储。
4
5  MVP 阶段采用如下边界：
6
7  1. DocForgeState
8      - 表示当前软著生成任务的业务状态；
9      - 用于 LangGraph 节点之间传递状态；
10     - 保存 source_id、file_path、qdrant_collection、evidence_id、draft_id、
      audit_report_id 等指针；
11     - 不保存完整原始文档；
12     - 不保存完整截图；
13     - 不保存完整 DOCX；
14     - 不保存大段原始资料全文。
15
16  2. state.json
17     - 是 DocForgeState 的本地持久化文件；

```



```

18     - 路径为 data/runs/{run_id}/state.json；
19     - 用于任务恢复、调试、排查问题；
20     - MVP 阶段优先使用 JSON，不默认启用 SQLite。
21
22 3. Qdrant
23     - 只保存可检索 Evidence Unit；
24     - 只保存 EvidenceItem 的向量和 payload；
25     - 不保存 LangGraph 工作流状态；
26     - 不保存完整原始资料；
27     - 不保存最终 DOCX。
28
29 4. 本地文件系统
30     - 保存用户上传的原始文件；
31     - 保存解析后的文本；
32     - 保存中间草稿；
33     - 保存内部 audit_report；
34     - 保存最终导出的 docx。
35
36 5. LangGraph Runtime State
37     - 只负责工作流运行时的节点传参和状态流转；
38     - MVP 阶段不把 LangGraph checkpoint 作为唯一状态源；
39     - 后续如果需要断点续跑、多进程恢复、长任务恢复，再引入 LangGraph checkpoint。
40
41 结论：
42
43 MVP 阶段的主状态源是：
44
45 data/runs/{run_id}/state.json
46
47 LangGraph 负责调度，state.json 负责业务状态持久化，Qdrant 负责证据语义检索。

```

用LangGraph追踪过程，看效果的一个关键对象

注意：URL 相关字段为 Phase 2 预留字段，MVP 不使用。我们当前MVP版本去掉了对用户给一个URL我们去自动爬网页的这个功能

代码块

```

1  from typing import TypedDict, Literal, Optional, Any
2
3
4  WorkflowStatus = Literal[
5      "CREATED",
6      "MATERIAL_UPLOADED",

```

```
7      "SOURCE_PARSED",
8      "REFERENCE_STYLE_ANALYZED",
9      "PRODUCT_UNDERSTOOD",
10     "EVIDENCE_MAPPED",
11     "DIAGNOSED",
12     "TEMPLATE_RECOMMENDED",
13     "USER_CONFIRM_REQUIRED",
14     "USER_CONFIRMED",
15     "PLAN_FROZEN",
16     "OUTLINE_CREATED",
17     "PLAN_GATE_REVIEWING",
18     "PLAN_GATE_PASSED",
19     "PLAN_GATE_FAILED",
20     "DRAFT_V1_CREATED",
21     "AUDIT_V1_COMPLETED",
22     "DRAFT_V2_CREATED",
23     "AUDIT_V2_COMPLETED",
24     "DRAFT_V3_CREATED",
25     "AUDIT_V3_COMPLETED",
26     "DRAFT_GATE_PASSED",
27     "REVISION_REQUIRED",
28     "FINAL_EXPORTED",
29     "RISK_EXPORTED",
30     "FAILED"
31 ]
32
33
34 NextAction = Literal[
35     "ingest_materials",
36     "parse_sources",
37     "analyze_reference_style",
38     "understand_product",
39     "extract_evidence",
40     "diagnose_software_type",
41     "recommend_template",
42     "ask_human_confirmation",
43     "freeze_doc_plan",
44     "create_outline",
45     "run_plan_quality_gate",
46     "ask_missing_information",
47     "write_draft",
48     "bind_screenshots",
49     "audit_draft",
50     "revise_draft",
51     "export_final_doc",
52     "export_risk_doc",
53     "stop"
```

```
54 ]
55
56
57 class SourceItem(TypedDict, total=False):
58     source_id: str
59
60     # 业务来源类型: 说明这份资料在业务上是什么
61     source_type: Literal[
62         "reference_soft_copyright_doc",
63         "product_intro_doc",
64         "prd",
65         "hld",
66         "detailed_design_doc",
67         "screenshot",
68         "user_note",
69         "product_url",    # Phase 2 预留, MVP 不使用
70         "other",
71     ]
72
73     # 文件格式类型: 说明这份资料的实际文件格式是什么
74     file_type: Literal[
75         "docx",
76         "pdf",
77         "md",
78         "txt",
79         "html",
80         "png",
81         "jpg",
82         "jpeg",
83         "webp",
84         "none",
85         "other",
86     ]
87
88     corpus_type: Literal[
89         "reference_style",
90         "product_evidence",
91     ]
92
93     allowed_usage: Literal[
94         "style_only",
95         "factual_evidence",
96     ]
97
98     file_name: Optional[str]
99     file_path: Optional[str]
100
```

```
101     # Phase 2 预留字段, MVP 阶段不使用
102     url: Optional[str]
103
104     is_reference_source: bool
105     is_product_source: bool
106
107     uploaded_at: str
108
109     parse_status: Literal[
110         "pending",
111         "parsed",
112         "failed",
113         "skipped",
114     ]
115
116     parse_error: Optional[str]
117     notes: Optional[str]
118
119     metadata: dict
120
121
122 class ParsedAsset(TypedDict, total=False):
123     asset_id: str
124     source_id: str
125     asset_type: Literal[
126         "text",
127         "image",
128         "table",
129         "page",
130         "url_page",
131         "screenshot",
132         "metadata"
133     ]
134     title: Optional[str]
135     summary: Optional[str]
136     extracted_text_ref: Optional[str]
137     image_ref: Optional[str]
138     page_number: Optional[int]
139     confidence: float
140
141
142 class CrawledPage(TypedDict, total=False):
143     page_id: str
144     source_id: str
145     url: str
146     title: Optional[str]
147     page_type: Optional[str]
```

```
148     extracted_text_ref: Optional[str]
149     screenshot_id: Optional[str]
150     navigation_items: list[str]
151     buttons: list[str]
152     forms: list[dict]
153     tables: list[dict]
154     crawl_status: Literal["success", "auth_required", "failed", "skipped"]
155     confidence: float
156     risk_notes: list[str]
157
158
159 class ProductSourceProfile(TypedDict, total=False):
160     uploaded_document_ids: list[str]
161     uploaded_screenshot_ids: list[str]
162
163     detected_navigation: list[str]
164     detected_page_titles: list[str]
165     detected_buttons: list[str]
166     detected_tables: list[dict]
167     detected_forms: list[dict]
168     detected_workflows: list[dict]
169
170     # Phase 2 预留字段, MVP 不做
171     product_url_ids: list[str]
172     crawled_pages: list[dict]
173     captured_screenshot_ids: list[str]
174     failed_urls: list[str]
175     auth_required_pages: list[str]
176     crawl_risk_notes: list[str]
177
178
179 class ReferenceStyleProfile(TypedDict, total=False):
180     common_chapter_structure: list[dict]
181     writing_style: str
182     screenshot_usage_pattern: str
183     operation_step_pattern: str
184     reusable_outline_pattern: list[dict]
185     prohibited_content_warning: list[str]
186
187
188 class ProductProfile(TypedDict, total=False):
189     product_positioning: str
190     target_users: list[str]
191     core_modules: list[str]
192     core_workflows: list[dict]
193     business_objects: list[str]
194     page_list: list[str]
```

```
195     feature_list: list[str]
196     technical_keywords: list[str]
197     industry_keywords: list[str]
198     uncertain_features: list[str]
199
200
201 class ProductFact(TypedDict, total=False):
202     fact_id: str
203     fact_type: Literal[
204         "software_name",
205         "version",
206         "module",
207         "feature",
208         "workflow",
209         "page",
210         "technical_environment",
211         "permission",
212         "data_object",
213         "other"
214     ]
215     content: str
216     source_ids: list[str]
217     confidence: float
218     is_confirmed: bool
219
220
221
222 # EvidenceItem 修正版
223 class EvidenceItem(TypedDict, total=False):
224     evidence_id: str
225     source_id: str
226
227     # 业务来源类型, 必须继承 SourceItem.source_type
228     source_type: Literal[
229         "reference_soft_copyright_doc",
230         "product_intro_doc",
231         "prd",
232         "hld",
233         "detailed_design_doc",
234         "screenshot",
235         "user_note",
236         "product_url",    # Phase 2 预留, MVP 不使用
237         "other",
238     ]
239
240     # 文件格式类型, 必须继承 SourceItem.file_type
241     file_type: Literal[
```

```
242         "docx",
243         "pdf",
244         "md",
245         "txt",
246         "html",
247         "png",
248         "jpg",
249         "jpeg",
250         "webp",
251         "none",
252         "other",
253     ]
254
255     evidence_type: Literal[
256         "product_document",
257         "product_screenshot",
258         "user_confirmation",
259         "reference_style_only",
260         "product_url_page",    # Phase 2 预留, MVP 不使用
261     ]
262
263     corpus_type: Literal[
264         "reference_style",
265         "product_evidence",
266     ]
267
268     allowed_usage: Literal[
269         "style_only",
270         "factual_evidence",
271     ]
272
273     evidence_strength: Literal[
274         "strong",
275         "medium",
276         "weak",
277         "not_allowed_as_fact",
278     ]
279
280     function_name: Optional[str]
281     related_module: Optional[str]
282     related_chapter: Optional[str]
283
284     content_ref: Optional[str]
285     summary: Optional[str]
286     extracted_facts: List[dict]
287     tags: List[str]
288
```

```

289     source_location: Optional[str]
290     location: Optional[str]
291
292     screenshot_id: Optional[str]
293
294     confidence: float
295
296     is_confirmed: bool
297     needs_human_confirmation: bool
298
299     notes: Optional[str]
300
301     metadata: dict
302
303
304 class DiagnosisResult(TypedDict, total=False):
305     primary_type: str
306     primary_type_confidence: float
307     business_objects: list[str]
308     enhancement_tags: list[str]
309     recommended_doc_style: str
310     alternative_doc_styles: list[str]
311     diagnosis_reasons: list[str]
312     risk_notes: list[str]
313
314
315 class TemplateStrategy(TypedDict, total=False):
316     base_template_id: str
317     base_template_name: str
318     enhancement_pack_ids: list[str]
319     excluded_template_ids: list[str]
320     recommended_chapters: list[str]
321     optional_chapters: list[str]
322     risk_chapters: list[str]
323     recommendation_reason: list[str]
324
325
326 class HumanConfirmation(TypedDict, total=False):
327     confirmation_id: str
328     confirmation_type: Literal[
329         "template_strategy",
330         "missing_information",
331         "feature_confirmation",
332         "screenshot_permission",
333         "final_export"
334     ]
335     prompt: str

```



```

336     options: List[str]
337     user_choice: Optional[str]
338     user_notes: Optional[str]
339     confirmed_at: Optional[str]
340     status: Literal["pending", "confirmed", "rejected", "overridden"]
341
342
343     class FrozenDocPlan(TypedDict, total=False):
344         plan_id: str
345         project_id: str
346         plan_version: str
347         locked_status: Literal["draft", "locked", "unlocked_for_revision"]
348         locked_at: Optional[str]
349         locked_by: Literal["orchestrator", "human", "system"]
350
351         software_identity: dict
352         diagnosis_snapshot: DiagnosisResult
353         template_decision: dict
354
355         chapter_policy: dict
356         feature_policy: dict
357         evidence_policy: dict
358         screenshot_policy: dict
359         writing_policy: dict
360         quality_gate_policy: dict
361         downstream_permissions: dict
362
363
364     class DocumentOutline(TypedDict, total=False):
365         outline_id: str
366         based_on_plan_id: str
367         chapters: List[dict]
368         required_evidence: List[dict]
369         required_screenshots: List[str]
370         estimated_pages: Optional[int]
371
372
373     class SectionPlan(TypedDict, total=False):
374         section_id: str
375         chapter_title: str
376         section_title: str
377         writing_goal: str
378         required_evidence_ids: List[str]
379         required_screenshot_ids: List[str]
380         writing_constraints: List[str]
381
382

```

```
383 class ScreenshotBinding(TypedDict, total=False):
384     screenshot_id: str
385     source_id: str
386     suggested_chapter: str
387     suggested_section: str
388     screenshot_description: str
389     confidence: float
390     status: Literal["bound", "missing", "candidate", "rejected"]
391
392
393 class MissingInformationItem(TypedDict, total=False):
394     item_id: str
395     question: str
396     reason: str
397     severity: Literal["blocker", "major", "minor"]
398     related_chapter: Optional[str]
399     related_feature: Optional[str]
400     status: Literal["pending", "answered", "ignored"]
401
402
403 class DraftVersion(TypedDict, total=False):
404     draft_id: str
405     version_label: Literal["v1", "v2", "v3", "risk"]
406     based_on_plan_id: str
407     based_on_outline_id: str
408     content_ref: str
409     created_at: str
410     revision_notes: Optional[str]
411     source_audit_report_id: Optional[str]
412
413
414 class AuditIssue(TypedDict, total=False):
415     issue_id: str
416     issue_type: Literal[
417         "plan_violation",
418         "unsupported_feature",
419         "future_as_current",
420         "reference_contamination",
421         "screenshot_mismatch",
422         "version_inconsistency",
423         "missing_environment",
424         "weak_operation_steps",
425         "advertising_language",
426         "legal_risk",
427         "other"
428     ]
429     severity: Literal["blocker", "major", "minor", "suggestion"]
```

```
430     location: Optional[str]
431     description: str
432     suggested_fix: str
433
434
435 class AuditReport(TypedDict, total=False):
436     audit_report_id: str
437     draft_id: str
438     audit_round: int
439     issues: List[AuditIssue]
440     score: int
441     blocker_count: int
442     major_count: int
443     minor_count: int
444     passed: bool
445     recommended_next_action: NextAction
446
447
448 class QualityGateReport(TypedDict, total=False):
449     gate_id: str
450     gate_type: Literal["plan_quality_gate", "draft_quality_gate"]
451     target_id: str
452     passed: bool
453     blocker_issues: List[str]
454     major_issues: List[str]
455     minor_issues: List[str]
456     required_user_questions: List[str]
457     next_action: NextAction
458
459
460 class ExportResult(TypedDict, total=False):
461     export_id: str
462     export_type: Literal["final", "risk"]
463     docx_path: Optional[str]
464     pdf_path: Optional[str]
465     risk_report_path: Optional[str]
466     exported_at: Optional[str]
467     export_notes: List[str]
468
469
470 class StateTransitionLog(TypedDict, total=False):
471     from_status: WorkflowStatus
472     to_status: WorkflowStatus
473     node_name: str
474     reason: str
475     created_at: str
476
```

```
477
478
479
480 class DocForgeState(TypedDict, total=False):
481     # 基础任务信息
482     run_id: str
483     project_id: str
484     project_name: str
485     target_doc_type: str
486     target_product_name: str
487     user_goal: str
488     output_requirements: dict
489
490     # WF-02 状态机控制
491     workflow_status: WorkflowStatus
492     next_action: NextAction
493     status_history: list[StateTransitionLog]
494
495     # 资料登记表: 不直接存全文
496     source_registry: list[SourceItem]
497     reference_source_ids: list[str]
498     product_source_ids: list[str]
499     screenshot_source_ids: list[str]
500     product_url_source_ids: list[str]
501
502     # 资料解析结果
503     parsed_assets: list[ParsedAsset]
504     qdrant_collection: str
505
506     # URL / 网页 / 截图采集结果
507     product_source_profile: ProductSourceProfile
508
509     # 参考软著风格画像
510     style_profile: ReferenceStyleProfile
511
512     # 产品理解结果
513     product_profile: ProductProfile
514     product_facts: list[ProductFact]
515     evidence_map: list[EvidenceItem]
516
517     # 软件类型诊断与模板策略
518     diagnosis_result: Optional[DiagnosisResult]
519     template_strategy: Optional[TemplateStrategy]
520
521     # 用户确认
522     human_confirmations: list[HumanConfirmation]
523     pending_human_questions: list[str]
```

```
524     missing_information: list[MissingInformationItem]
525
526     # 冻结计划: WF-02 核心控制对象
527     frozen_doc_plan: Optional[FrozenDocPlan]
528
529     # 大纲和写作计划
530     outline: Optional[DocumentOutline]
531     section_plan: list[SectionPlan]
532
533     # 截图绑定
534     screenshot_map: list[ScreenshotBinding]
535
536     # Plan Quality Gate
537     plan_quality_gate_report: Optional[QualityGateReport]
538     plan_quality_gate_passed: bool
539
540     # 草稿版本
541     draft_versions: list[DraftVersion]
542     current_draft_id: Optional[str]
543     current_draft_version: Optional[str]
544
545     # Draft Audit / Draft Quality Gate
546     audit_reports: list[AuditReport]
547     draft_quality_gate_reports: list[QualityGateReport]
548     current_score: Optional[int]
549     blocker_issues: list[str]
550     major_issues: list[str]
551     minor_issues: list[str]
552
553     # 循环控制
554     revision_round: int
555     max_revision_round: int
556
557     # 导出结果
558     export_result: Optional[ExportResult]
559     final_doc_path: Optional[str]
560     final_pdf_path: Optional[str]
561     risk_report_path: Optional[str]
562
563     # 异常与调试
564     errors: list[dict]
565     warnings: list[str]
```

备注:

State 里不应该塞完整 docx、完整截图。State 里应该放:

代码块文件路径

```
2 source_id
3 Qdrant collection 名
4 检索结果 id
5 提取出的结构化摘要
6 当前工作流结果
```

- 原始资料放在文件系统。
- 可检索内容放在 Qdrant。
- 当前任务进度放在 State。

这三者必须分开。

备注:

资料在 State 里的表示方式，这点必须严格。比如老板给的外部软著、我们自己的产品文档，图片截图：

代码块

```
1 # =====
2 # SourceItem 示例 (Python Dict 格式)
3 # =====
4
5 # 1. 外部参考软著示例
6 source_item_reference: SourceItem = {
7     "source_id": "src_ref_001",
8     "source_type": "reference_soft_copyright_doc",
9     "file_type": "docx",
10    "corpus_type": "reference_style",
11    "allowed_usage": "style_only",
12    "file_name": "某外部软著.docx",
13    "file_path": "data/runs/{run_id}/sources/reference/某外部软著.docx",
14    "url": None,
15    "is_reference_source": True,
16    "is_product_source": False,
17    "uploaded_at": "2026-06-05T10:00:00",
18    "parse_status": "pending",
19    "parse_error": None,
20    "notes": "外部参考软著，只允许学习目录结构、章节写法、配图方式和语言风格。",
21    "metadata": {}
22 }
23
24 # 2. 我方产品 HLD 示例
25 source_item_hld: SourceItem = {
```

```

26     "source_id": "src_prod_001",
27     "source_type": "hld",
28     "file_type": "docx",
29     "corpus_type": "product_evidence",
30     "allowed_usage": "factual_evidence",
31     "file_name": "Refinar_HLD.docx",
32     "file_path": "data/runs/{run_id}/sources/product/Refinar_HLD.docx",
33     "url": None,
34     "is_reference_source": False,
35     "is_product_source": True,
36     "uploaded_at": "2026-06-05T10:05:00",
37     "parse_status": "pending",
38     "parse_error": None,
39     "notes": "我方产品资料，可作为产品事实来源。",
40     "metadata": {}
41 }
42
43 # 3. 产品截图示例
44 source_item_screenshot: SourceItem = {
45     "source_id": "src_prod_002",
46     "source_type": "screenshot",
47     "file_type": "png",
48     "corpus_type": "product_evidence",
49     "allowed_usage": "factual_evidence",
50     "file_name": "dataset_page.png",
51     "file_path": "data/runs/{run_id}/sources/screenshots/dataset_page.png",
52     "url": None,
53     "is_reference_source": False,
54     "is_product_source": True,
55     "uploaded_at": "2026-06-05T10:10:00",
56     "parse_status": "pending",
57     "parse_error": None,
58     "notes": "产品页面截图，属于强证据。",
59     "metadata": {}
60 }

```

这套设计能保证：**参考资料永远只能影响写法，不能污染事实。**

3、FrozenDocPlan对象

下面是：FrozenDocPlan 数据结构 v1.0

以你们 Refinar 这种产品为例，FrozenDocPlan 大概是这样（示例）：

代码块 frozenDocPlan

```
2
3   plan_id: FDP-20260604-001
4   project_id: SR-REFINAR-001
5   plan_version: v1
6   locked_status: locked
7
8   software_identity:
9     software_name: Ragentic Refinar 数据资产管理平台
10    version: V1.0
11    target_doc_name: Ragentic Refinar 数据资产管理平台用户操作手册
12
13  diagnosis_snapshot:
14    primary_type: Web/SaaS 平台
15    business_objects:
16      - 数据资产
17      - 数据集
18    enhancement_tags:
19      - 数据平台
20      - AI 平台轻量增强
21      - 汽车工业软件
22    recommended_doc_style: 用户操作手册型软著文档
23
24  template_decision:
25    selected_doc_style: 用户操作手册型
26    base_template_id: TEMPLATE_WEB_SAAS_USER_MANUAL
27    enhancement_pack_ids:
28      - PACK_DATA_PLATFORM
29      - PACK_AI_LIGHT
30      - PACK_PERMISSION_MANAGEMENT
31    decision_source: user_confirmed_agent_recommendation
32
33  chapter_policy:
34    fixed_level_1_chapters:
35      - 引言
36      - 软件概述
37      - 运行环境
38      - 登录与首页
39      - 数据资产管理
40      - 数据集管理
41      - 数据导入与处理
42      - 质量检查与报告
43      - 训练/模型相关功能
44      - 用户与权限管理
45      - 常见问题与附录
46    chapter_order_locked: true
47    level_1_editable: false
```



```
48     level_2_editable: true
49
50     feature_policy:
51         allowed_features:
52             - 用户登录
53             - 首页看板
54             - 数据资产列表
55             - 数据集管理
56             - 数据导入
57             - 数据质检
58             - 质量报告
59             - 用户权限管理
60         uncertain_features:
61             - 在线训练
62             - 模型评测
63         prohibited_features:
64             - 自动驾驶闭环仿真
65             - 完整 PLM 流程管理
66         future_features:
67             - 对外数据交易
68             - 多组织复杂审批流
69
70     evidence_policy:
71         allowed_evidence_sources:
72             - 自有产品文档
73             - 自有 PRD 文档
74             - 自有 HLD 文档
75             - 产品截图
76             - 文档内提取图片
77             - 用户确认信息
78         prohibited_evidence_sources:
79             - 参考软著中的产品功能描述
80         reference_doc_usage_scope:
81             - 目录结构
82             - 写作风格
83             - 截图组织方式
84
85     screenshot_policy:
86         screenshot_required: true
87         minimum_screenshot_count: 8
88
89         required_screenshot_types:
90             - 登录页
91             - 首页
92             - 数据资产列表页
93             - 数据集详情页
94             - 数据导入页
```

```
95     - 质检报告页
96     - 用户管理页
97     - 权限配置页
98
99     user_upload_allowed: true
100     document_image_extract_allowed: true
101     screenshot_auto_binding_allowed: true
102
103     # MVP 不做 URL 网页自动截图
104     site_auto_capture_allowed: false
105
106     missing_screenshot_handling: ask_user_or_use_textual_description
107
108     writing_policy:
109         writing_style: 用户操作手册风格
110         tense_rule: 当前版本已实现功能用陈述语气
111         forbidden_phrases:
112             - 行业领先
113             - 全球首创
114             - 完全自主可控
115             - 颠覆性
116         no_future_as_current_rule: true
117
118     quality_gate_policy:
119         max_revision_rounds: 3
120         pass_condition:
121             - no_blocker
122             - major_issue_count <= 2
123         risk_export_condition:
124             - reached_max_revision_rounds
125             - still_has_major_or_blocker_issues
```

4、最终产出物

1. 《软件著作权文档》.docx
2. ~~《软著生成审计报告》.docx 或 .md~~（这个先不做了，浪费Token）

解决DOCX 格式会乱的问题！

生成出来的Docx格式会乱，除非你做模板控制。不能让 AI 直接“生成 docx”。

正确方式是：

代码块

- 1 AI 生成结构化章节内容
- 2 ↓
- 3 系统用 python-docx 按固定模板写入 DOCX

也就是说，AI 不负责排版。AI 只输出结构化内容，例如：

代码块

```
1  {
2    "chapters": [
3      {
4        "title": "1 引言",
5        "sections": [
6          {
7            "title": "1.1 编写目的",
8            "content": "....."
9          }
10       ]
11     }
12   ]
13 }
```

然后 ExportAgent 用 `python-docx` 控制：

代码块

- 1 一级标题样式
- 2 二级标题样式
- 3 正文样式
- 4 页边距
- 5 字体
- 6 行距
- 7 表格样式
- 8 图片宽度
- 9 页码
- 10 目录占位

MVP 不做高级排版，但要做基础统一格式。

建议基础格式：

代码块

- 1 纸张：A4
- 2 中文字体：宋体

- 3 英文字体: Times New Roman
- 4 正文字号: 小四
- 5 一级标题: 黑体三号
- 6 二级标题: 黑体四号
- 7 正文行距: 1.5 倍
- 8 图片宽度: 不超过页面宽度
- 9 图片标题: 图 1-1 xxx

这就够了。

设计时必须注意

1、资料必须分成两类，而且要做“用途隔离”

你现在说的两类资料非常关键：

第一类是**外部参考资料**（可能是别的公司的软著）。

它只能用于学习：

代码块

- 1 目录结构
- 2 章节顺序
- 3 标题写法
- 4 描述方式
- 5 配图方式
- 6 文档语气
- 7 软著常见表达

它**绝对不能**用于学习产品事实。

也就是说，别人公司软著里写了“权限管理、流程编排、三维仿真、数据看板”，不能因为参考文档里有这些，就写进你们自己的产品里。

第二类是**我们自己的产品资料**。

它才是事实来源，包括：

代码块

- 1 产品需求文档 PRD
- 2 产品 HLD 顶层设计文档

- 3 产品说明文档
- 4 产品网页截图 png/jpeg
- 5 系统原型截图
- 6 后端关键代码片段
- 7 接口文档
- 8 README
- 9 数据库表结构
- 10 部署说明

这两类资料必须在系统里物理隔离或逻辑隔离。最小方案可以放在两个目录里；更好的方案是在入库时给每个资料打 `corpus_type`：

代码块

- 1 `reference_style`: 只允许学习写法，不允许提供产品事实
- 2 `product_evidence`: 允许作为产品事实依据

这是整个系统的生命线。否则这个 Agent 很容易把“别人家的产品能力”偷渡进“我们公司的软著”里，最后文档看起来很像样，但事实是假的。

2、输入资料格式要重新设计：不是“文档解析”，而是“多模态资料理解”

上一版说 docx/txt 太窄了。新版应该把输入层定义成：

代码块

- 1 Source Ingestion Layer
- 2 资料接入层

支持这些资料：

代码块

- 1 1. 文档类
- 2 docx / pdf / md / txt / html
- 3
- 4 2. 图片类
- 5 png / jpg / jpeg
- 6 主要用于产品网页截图、原型图、系统界面截图
- 7
- 8 3. 项目说明类
- 9 README / API 文档 / HLD / PRD / 原型说明

关键是：**所有格式最后都要转换成统一的 Evidence Unit。**

也就是不管原始资料是截图、docx、代码还是 markdown，系统最后都要变成类似这种结构：

代码块

```
1  Evidence Unit
2
3  - evidence_id: 证据编号
4  - source_file: 来自哪个文件
5  - source_type: docx / image / code / pdf / md
6  - corpus_type: reference_style / product_evidence
7  - allowed_usage: style_only / factual_evidence
8  - content: 提取出来的正文或描述
9  - summary: 摘要
10 - extracted_facts: 提取出的事实点
11 - tags: 功能模块、技术架构、界面、流程、角色等标签
12 - confidence: 可信度
13 - location: 页码、段落、截图区域、代码行号等位置
```

这一步很重要。

你的系统真正需要的不是“把文件塞给大模型”，而是**把乱七八糟的资料变成可检索、可引用、可审计的证据单元。**

3、新增 FrozenDocPlan

用户确认之后，必须生成冻结方案。

后续 WriterAgent 只能按这个方案写。

没有 FrozenDocPlan，你们这个系统后面一定会乱。

这是控制软著文档质量的核心对象。

工程化目录：

代码块

```
1  docforge/
2    app/
3      main.py
4    graph/
5    agents/
6    models/
```

```
7     services/
8     storage/
9     exporters/
10    prompts/
11    config/
12    data/
13    runs/
14        {run_id}/
15        sources/
16            reference/
17            product/
18            screenshots/
19            urls/
20    parsed/
21    evidence/
22    drafts/
23    audits/
24    exports/
25    state.json
26    tests/
27    pyproject.toml
28    README.md
```

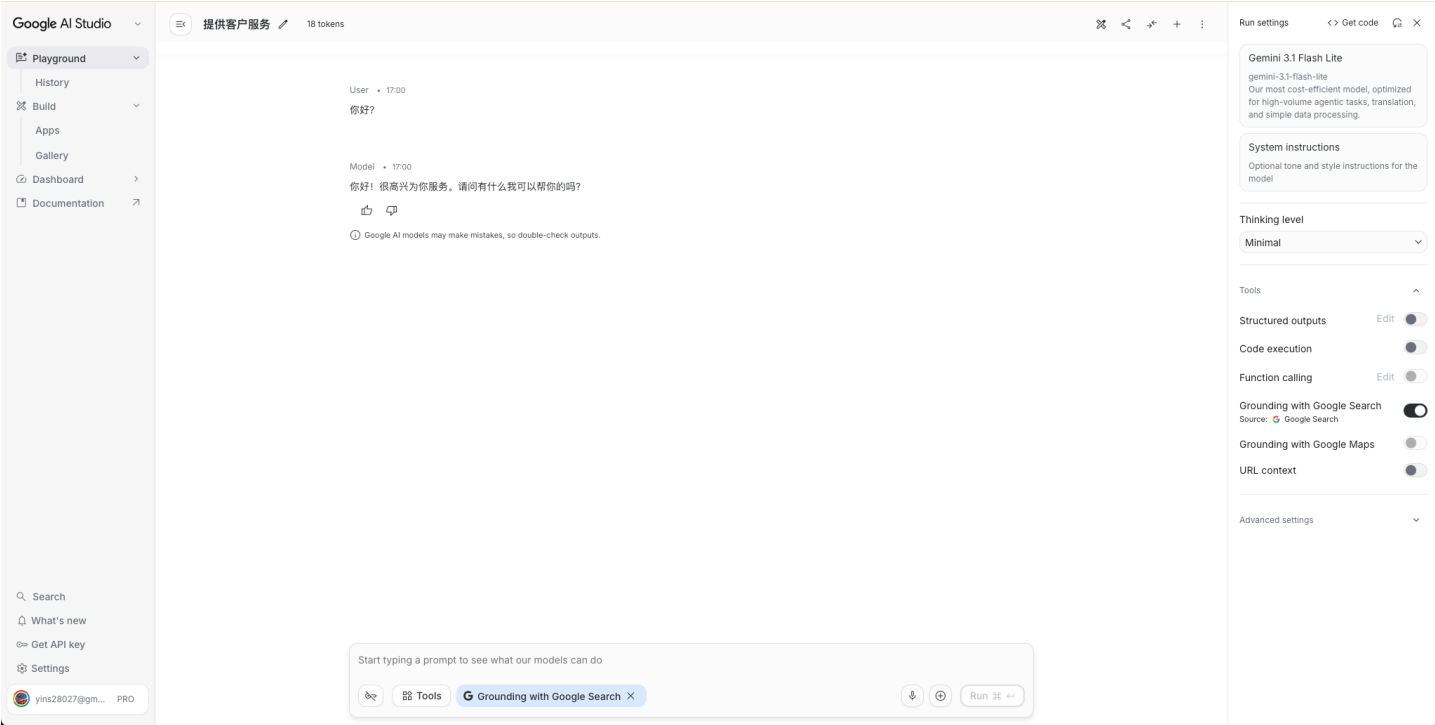
4、交互说明

用户进入界面后，会呈现一个跟Google AI Studio类似的界面：

左边：导航栏的区域，就是让用户存放【外部参考资料】，以及公司内部的【自家产品资料】的地方。

中间：就是AI和人对话的地方。

右边：让人去选择【例如：请选择你的产品更接近哪一类：】，这样的一些选项控制台，方便给AI更多的信息和上下文，以及一些约束的；当然也有让用户选择，你要下载的文档是要求是什么格式的，诸如此类的吧。



辅助AI知道自家产品是什么大类的，有助于AI建立初步的文章目录，这个可能交互过程如下：

代码块

1

请选择你的产品更接近哪一类：

2

3

A. SaaS / Web 平台

4

B. AI 平台

5

C. 数据平台

6

D. 我不知道，让 Agent 根据资料判断

系统逻辑应该是：

代码块

1

用户选择 A/B/C：

2

作为 SoftwareDiagnosisAgent 的 prior_hint

3

但 Agent 仍然要根据资料做诊断

4

5

用户选择 D：

6

Agent 完全根据资料判断

如果用户选择和 Agent 判断冲突，要展示出来：

代码块

1

你选择的是：AI 平台

- 2 Agent 根据资料判断：Web/SaaS 平台 + 数据平台增强
- 3 建议采用：用户操作手册型软著文档
- 4
- 5 是否按 Agent 推荐继续？
- 6 A. 按 Agent 推荐
- 7 B. 按我选择的 AI 平台
- 8 C. 查看差异原因

这才稳。不要让用户一选就直接锁死模板。

5、Qdrant MVP 使用规则

注意：EmbeddingProvider 使用Jina-embeddings-v3

1、部署方式

MVP 使用 Qdrant Local Persistent Mode，不使用远程 Qdrant，不要求 Docker 启动 Qdrant Server。

默认初始化方式：

代码块

```
1 from qdrant_client import QdrantClient
2
3 client = QdrantClient(path="data/qdrant")
```

说明：

1. 本地 Qdrant 数据持久化存放在 `data/qdrant`；
2. 不使用 `QdrantClient(":memory:")` 作为默认模式，因为内存模式在程序退出后数据会丢失；
3. 后续如果需要多用户、多进程、长期服务或更高性能，再切换到 Docker Qdrant Server；
4. Qdrant 只保存可检索 Evidence Unit，不保存 LangGraph workflow 状态。

2、Collection 命名

每次软著生成任务创建一个独立 collection：

代码块

```
1 docforge_{run_id}
```

例如：

代码块

```
1 docforge_20260604_153000_ab12
```

命名规则：

1. 全部小写；
2. 只使用字母、数字、下划线；
3. run_id 必须唯一；
4. 不同任务之间的证据必须隔离。

3、写入 Qdrant 的内容

每个 EvidenceItem 写入一条 point。

vector 内容来自：

代码块

```
1 summary + extracted_facts + tags
```

payload 至少包含：

代码块

```
1 evidence_id
2 source_id
3 source_type
4 corpus_type
5 allowed_usage
6 evidence_type
7 evidence_strength
8 function_name
9 related_module
10 related_chapter
11 summary
12 tags
13 source_location
14 confidence
```

```
15 is_confirmed
16 needs_human_confirmation
```

Qdrant 中不保存完整原始文档、不保存完整 DOCX、不保存完整截图。

原始文件保存在本地文件系统：

代码块

```
1 data/runs/{run_id}/sources/
```

结构化状态保存在：

代码块

```
1 data/runs/{run_id}/state.json
```

Qdrant 只用于语义检索证据。

4、检索规则

WriterAgent 查询产品事实时，必须使用过滤条件：

代码块

```
1 corpus_type = product_evidence
2 allowed_usage = factual_evidence
```

ReferenceStyleAgent 查询参考风格时，必须使用过滤条件：

代码块

```
1 corpus_type = reference_style
2 allowed_usage = style_only
```

AuditAgent 检查事实来源时，必须验证：

代码块

```
1 所有产品事实引用的 EvidenceItem 必须满足：
2 corpus_type = product_evidence
3 allowed_usage = factual_evidence
```

禁止 WriterAgent 在生成产品事实时使用：

代码块

```
1 corpus_type = reference_style
2 allowed_usage = style_only
```

5、top_k 设置

MVP 默认检索参数：

代码块

```
1 //默认
2 top_k = 8
3
4 几个细分场景的Top-k：
5 ReferenceStyleAgent: top_k = 6
6 ProductUnderstandingAgent: top_k = 10
7 WriterAgent: top_k = 8
8 AuditAgent: top_k = 12
```

不同场景可微调：

场景	top_k
ReferenceStyleAgent 提取参考风格	6
ProductUnderstandingAgent 理解产品功能	10
WriterAgent 写单个章节	8
AuditAgent 查证据	12

6、Payload Filter 原则

Qdrant 支持基于 payload 条件进行过滤；在语义检索中，必须通过 payload filter 区分 reference_style 和 product_evidence，避免参考软著污染产品事实。Qdrant 官方文档也说明，搜索和检索时可以对 payload 设置过滤条件，并且建议对需要过滤的字段创建 payload index 以提升性能。

MVP 应至少对以下 payload 字段创建索引：

代码块

```
1 corpus_type
2 allowed_usage
3 source_type
4 evidence_type
```

```
5 function_name
6 related_module
7 tags
```

7、后续切换到 Docker Qdrant Server 的条件

当出现以下情况时，再考虑从 Local Persistent Mode 切换到 Docker Qdrant Server：

1. 单次任务证据数量超过 20,000 条；
2. 需要多进程或多人同时访问；
3. Streamlit 前端和 Agent 后端拆成独立服务；
4. 需要长期运行的服务化部署；
5. 本地模式出现明显性能瓶颈；
6. 需要 Qdrant Dashboard 或运维管理能力。

MVP 阶段不需要先上 Docker Qdrant Server。

6、Streamlit 上传文件必须立即落盘

Streamlit 的 `st.file_uploader` 默认上传文件大小有限制，并且上传内容会进入 Python 内存，不是自动保存到磁盘。

所以 Sprint 1 必须做：

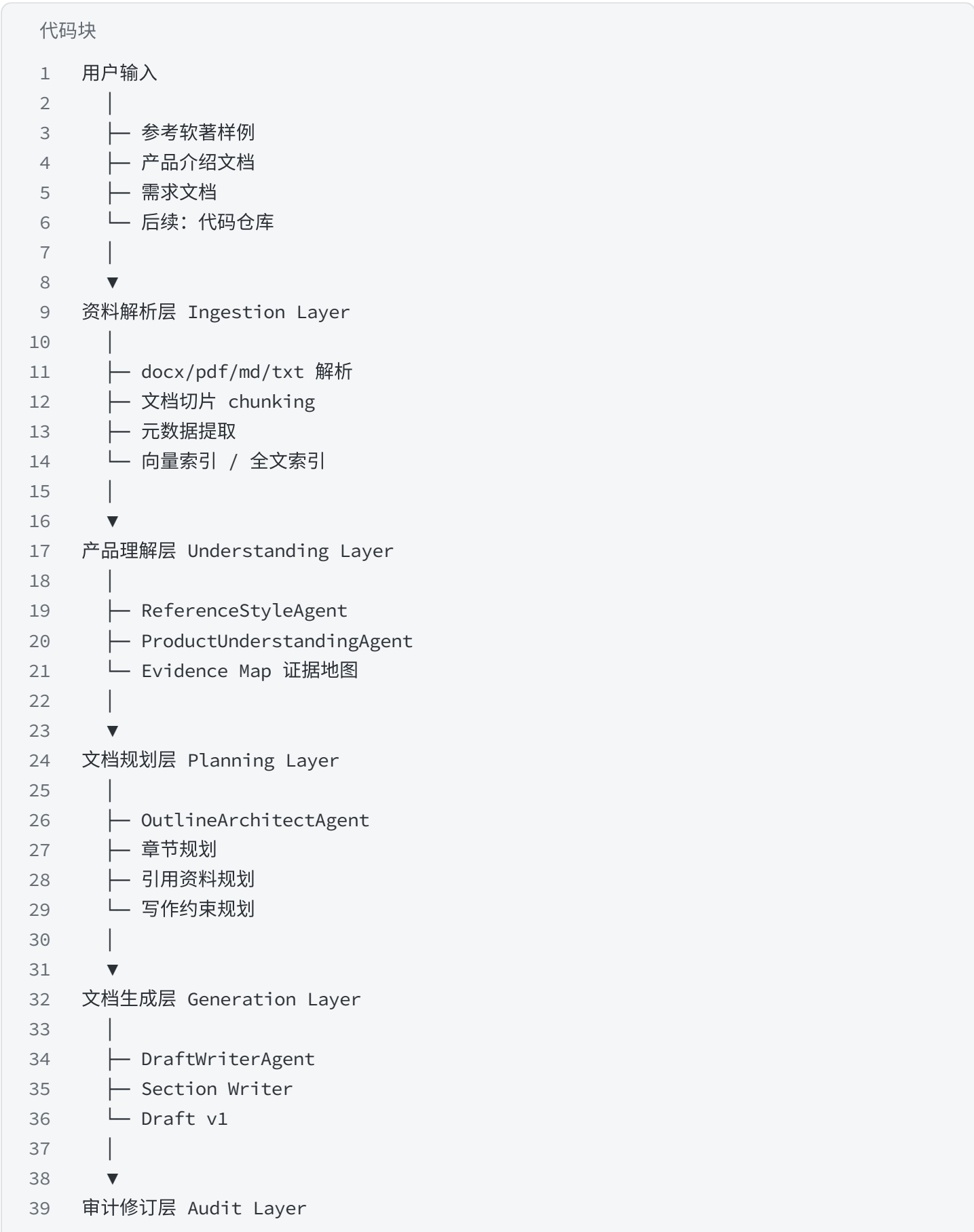
代码块

```
1 用户上传文件
2  ↓
3  立刻写入 data/runs/{run_id}/sources/
4  ↓
5  生成 SourceItem
6  ↓
7  写入 state.json
```

不要让 Agent 直接消费 Streamlit 的 UploadedFile 对象

附录

业务架构



```
40 |
41 |─ AuditAndRevisionAgent
42 |─ 格式审计
43 |─ 事实审计
44 |─ 完整性审计
45 |─ 风格审计
46 |─ Draft v2 / v3
47 |
48 ▼
49 导出层 Export Layer
50 |─ DOCX
51 |─ 内部运行数据：
52 |   |─ state.json: MVP 必须实现，用于保存任务状态
53 |   |─ evidence_map: MVP 必须实现，可存在 State / Qdrant payload 中
54 |   |─ audit_report: MVP 必须实现为内部对象，用于驱动 v2/v3 修订，不作为用户下载产
    物
```



经过6轮需求文档的审计，校对，修正，目前的《初版需求描述》已经进入了可以正式开发的阶段，下面是一个《交接说明》，用于发给大模型，让大模型开始开启本次的开发工作，包括：确定项目开发的架构，确定技术栈，划分Sprint，写出每个Sprint的提示词给Cursor等

暂存一个《初版需求描述》文档的快照（Date：2026年6月5号）：



 初版需求描述 (1).md

墨衡 DocForge 项目交接说明

1. 本次对话结论

本轮已经完成《墨衡 DocForge》从产品设想到 MVP 工程落地前的设计收敛。

最终判断：

代码块

```
1    B：当前设计文档已经具备工程落地的基本要求，可以用 Cursor 开始落地工程。
```

但进入开发时要注意：不是一次性让 Cursor 写完整系统，而是要按 Sprint 拆分，逐步落地。

2. 当前项目定位

项目名称：

代码块

1 墨衡 DocForge

产品定义：

代码块

1 墨衡 DocForge 是一个面向软件著作权文档生成的多智能体文档生产系统。它根据外部参考软著样例、自家公司产品资料、需求文档，自动完成产品理解、文档规划、初稿生成、审计评分、多轮修订和最终导出。

核心价值：

代码块

1 帮助用户基于参考软著写法和自有产品资料，生成一份可编辑的《软件著作权文档》.docx。

系统本质：

代码块

1 不是普通文档生成器，而是一个具备证据隔离、类型诊断、模板冻结、审计修订、DOCX 导出的软著文档生产系统。

3. MVP 范围

MVP 要做：

代码块

```
1 1. 读取参考软著样例
2   - 支持 docx / pdf / md / txt
3   - 只用于学习目录结构、章节写法、配图方式、语言风格
4   - 不允许作为产品事实来源
5
6 2. 读取目标产品资料
7   - 支持产品介绍文档、PRD、HLD、产品截图等
8   - 作为产品事实来源
9
10 3. 自动生成一版软著文档草稿
11
```

```
12  4. 对草稿进行最多 3 轮“审计-修订”循环
13    - v1 / v2 / v3
14    - 通过则导出最终版
15    - 不通过则导出风险版 DOCX
16
17  5. 做本地轻量前端界面
18    - 使用 Streamlit
19    - 支持文件上传、产品类型选择、方案确认、结果下载
20
21  6. 最终用户下载产物只有一份：
22    - 《软件著作权文档》.docx
23    - 若 v3 仍未通过，则导出《软件著作权文档_风险版》.docx
```

MVP 不做：

代码块

```
1  1. 不接代码仓库
2  2. 不通过 URL 读取网页
3  3. 不做复杂权限
4  4. 不接公司知识库
5  5. 不做多格式高级排版
6  6. 不做多租户
7  7. 不做在线协作编辑
8  8. 不导出独立审计报告
9  9. 不导出 PDF
10 10. 不导出 Markdown
```

4. 已经拍板的技术栈

开发工具：

代码块

```
1  Cursor
```

运行环境：

代码块

```
1  macOS
2  Python 3.11 或 3.12
```

核心框架：

代码块

```
1 LangGraph
```

轻量前端：

代码块

```
1 Streamlit
```

模型层：

代码块

```
1 QwenProvider
2 DeepSeekProvider
```

Embedding：

代码块

```
1 Jina-embeddings-v3
```

RAG / 检索：

代码块

```
1 Qdrant Local Persistent Mode
```

文档解析：

代码块

```
1 docx: python-docx
2 pdf: pypdf / pymupdf
3 markdown / txt / html: 常规文本解析
4 图片: 视觉模型解析 + OCR 兜底
```

导出：

5. Qdrant 已确定方案

MVP 必须使用 Qdrant。

部署方式：

代码块

```
1 Qdrant Local Persistent Mode
```

默认初始化方式：

代码块

```
1 from qdrant_client import QdrantClient
2
3 client = QdrantClient(path="data/qdrant")
```

Collection 命名：

代码块

```
1 docforge_{run_id}
```

Qdrant 只保存 Evidence Unit，不保存 LangGraph workflow 状态。

WriterAgent 查询产品事实时，必须使用过滤条件：

代码块

```
1 corpus_type = product_evidence
2 allowed_usage = factual_evidence
```

ReferenceStyleAgent 查询参考风格时，必须使用过滤条件：

代码块

```
1 corpus_type = reference_style
2 allowed_usage = style_only
```

AuditAgent 必须检查所有产品事实引用的 EvidenceItem 是否满足：

代码块

```
1 corpus_type = product_evidence
2 allowed_usage = factual_evidence
```

默认 top_k：

代码块

```
1 top_k = 8
```

建议不同场景：

代码块

```
1 ReferenceStyleAgent: top_k = 6
2 ProductUnderstandingAgent: top_k = 10
3 WriterAgent: top_k = 8
4 AuditAgent: top_k = 12
```

6. 核心系统原则

必须坚持以下规则：

代码块

1. 外部软著只能提供写法参考，不能提供产品事实。
2. 我方产品资料才是事实来源。
3. 参考资料和产品资料必须通过 corpus_type / allowed_usage 隔离。
4. WriterAgent 写作时必须能追溯事实来源。
5. AuditAgent 必须检查 reference_style 污染。
6. State 只保存任务状态和证据指针，不保存大段原始资料。
7. Qdrant 保存可检索 Evidence Unit，不保存 workflow 状态。
8. 模型调用必须通过 Provider Adapter。
9. 审计循环采用评分阈值 + 最大轮次，不允许无限循环。
10. 用户确认模板后，FrozenDocPlan 必须冻结，后续 Agent 不得擅自修改一级目录。

7. 当前 Agent 团队设计

当前 Agent 团队编号：

代码块

1 AT-02：软著生成 Agent 团队 v2.0

核心 Agent：

代码块

- 1 1. OrchestratorAgent
 - 2 - 总指挥
 - 3 - 控制 LangGraph 状态机
 - 4 - 调度 Agent
 - 5 - 生成 FrozenDocPlan
 - 6 - 控制 v1/v2/v3 修订轮次
 - 7 - 触发最终导出
- 8
- 9 2. IntakeAgent
 - 10 - 创建软著项目
 - 11 - 采集软件名称、版本、著作权人、用途简述等基础信息
- 12
- 13 3. MaterialIngestionAgent
 - 14 - 接入资料
 - 15 - 区分参考软著资料、自有产品资料、产品截图、用户补充说明
- 16
- 17 4. ProductSourceAgent
 - 18 - 读取上传文档
 - 19 - 读取上传截图
 - 20 - 提取产品资料中的功能、页面、模块、流程
- 21
- 22 5. ReferenceStyleAgent
 - 23 - 分析参考软著的目录结构、章节粒度、截图插入方式、语言风格
 - 24 - 禁止提取对方产品事实
- 25
- 26 6. ProductUnderstandingAgent
 - 27 - 理解自有产品定位、目标用户、核心模块、业务对象、核心流程
- 28
- 29 7. EvidenceExtractorAgent
 - 30 - 抽取 Evidence Unit
 - 31 - 构建 EvidenceMap
 - 32 - 写入 Qdrant

```
33
34 8. SoftwareDiagnosisAgent
35   - 判断主类型、核心业务对象、增强标签、推荐文档写法
36
37 9. TemplateStrategyAgent
38   - 根据诊断结果推荐软著文档模板和章节策略
39
40 10. HumanConfirmGate
41   - 用户确认 Agent 推荐方案
42   - 用户可以按 Agent 推荐，也可以手动修改类型
43
44 11. OutlineAgent
45   - 基于 FrozenDocPlan 生成详细目录
46   - 不允许自由发明一级目录
47
48 12. WriterAgent
49   - 基于 FrozenDocPlan、DocumentOutline、EvidenceMap、ReferenceStyleProfile 写正
    文
50
51 13. ScreenshotBindingAgent
52   - 绑定截图到章节
53   - 生成截图说明
54   - 判断截图缺失或错配
55
56 14. AuditAgent
57   - 检查事实、目录、证据、截图、版本、参考污染、软著规范性
58
59 15. ExportAgent
60   - 使用 python-docx 导出最终 DOCX
```

8. workflow WF-02

当前 workflow 编号：

代码块

```
1  WF-02：软著文档生成 workflow v2.0
```

主流程：

代码块

```
1  OrchestratorAgent 启动任务
2  ↓
```

```
3  1. IntakeAgent: 创建软著项目
4    ↓
5  2. MaterialIngestionAgent: 资料接入与分类
6    ↓
7  3. ProductSourceAgent: 产品资料读取
8    ↓
9  4. ReferenceStyleAgent: 参考软著风格分析
10   ↓
11 5. ProductUnderstandingAgent: 自有产品理解
12   ↓
13 6. EvidenceExtractorAgent: 证据抽取与证据地图构建
14   ↓
15 7. SoftwareDiagnosisAgent: 软件类型诊断
16   ↓
17 8. TemplateStrategyAgent: 模板推荐与目录策略生成
18   ↓
19 9. HumanConfirmGate: 用户确认推荐方案
20   ↓
21 10. OrchestratorAgent: 生成并冻结 FrozenDocPlan
22   ↓
23 11. OutlineAgent: 基于 FrozenDocPlan 生成详细目录
24   ↓
25 12. PlanQualityGate: 生成前质量门禁
26   ↓
27 13. WriterAgent: 写 v1 初稿
28   ↓
29 14. ScreenshotBindingAgent: 截图绑定与章节配图
30   ↓
31 15. AuditAgent: 审查 v1
32   ↓
33 16. DraftQualityGate: 判断是否通过
34     └─ 通过: 进入导出
35     └─ 不通过: WriterAgent 根据审计意见写 v2 / v3
36   ↓
37 17. ExportAgent: 导出《软件著作权文档》.docx
```

如果 v3 仍不通过:

代码块

```
1  导出《软件著作权文档_风险版》.docx
```

不导出独立审计报告。

9. Quality Gate 设计

PlanQualityGate

作用：

代码块

- 1 判断是否可以开始正文生成。

它不打分，只做 checklist。

核心检查项：

代码块

- 1 1. 软件名称是否明确
- 2 2. 软件版本是否明确
- 3 3. 软件类型是否已确认
- 4 4. 文档模板是否已冻结
- 5 5. 一级目录是否已冻结
- 6 6. 是否区分参考资料和自有产品资料
- 7 7. 核心功能是否都有证据来源
- 8 8. 核心章节是否有截图或替代说明
- 9 9. 是否存在疑似未来规划功能
- 10 10. 是否存在必须向用户反问的缺失信息

DraftQualityGate

作用：

代码块

- 1 判断当前草稿是否可以导出。

使用 100 分扣分制：

代码块

- 1 $\text{score} = 100 - \text{blocker_count} * 30 - \text{major_count} * 8 - \text{minor_count} * 3$

通过条件：

代码块

- 1 1. score >= 85
- 2 2. blocker_count = 0
- 3 3. major_count <= 2
- 4 4. 必填章节完整
- 5 5. 产品事实均来自 product_evidence
- 6 6. reference_style 没有被当作产品事实使用
- 7 7. 一级目录没有违反 FrozenDocPlan
- 8 8. 软件名称和版本号全文一致
- 9 9. 没有将明确未来规划功能写成当前版本已实现功能

10. 审计问题分级

Blocker

代码块

- 1 1. 使用 reference_style 资料作为产品事实来源
- 2 2. 核心功能没有任何 product_evidence 支撑
- 3 3. 软件名称缺失或全文严重不一致
- 4 4. 软件版本缺失或全文严重不一致
- 5 5. 未生成 FrozenDocPlan 就开始生成正文
- 6 6. 一级目录违反 FrozenDocPlan
- 7 7. 将明确未来规划功能写成当前版本已实现功能
- 8 8. 文档核心章节缺失
- 9 9. WriterAgent 擅自新增 FrozenDocPlan 禁止写入的功能
- 10 10. 最终文档无法形成完整软著文档结构

Major

代码块

- 1 1. 核心章节有证据，但操作步骤明显不完整
- 2 2. 章节内容和证据弱匹配，存在表达过度或依据不足
- 3 3. FrozenDocPlan 要求必须配图的章节缺少截图，且没有替代说明
- 4 4. 术语前后不统一，影响理解
- 5 5. 运行环境描述过于模糊
- 6 6. 部分章节偏离用户操作手册型软著文档风格
- 7 7. 存在较多宣传式表达
- 8 8. 疑似将未来功能写成已实现功能，但证据不足以判定为 Blocker
- 9 9. 章节内容过薄

Minor

代码块

1. 个别句子表达不规范
2. 个别标题粒度不均衡
3. 个别截图说明不完整
4. 章节详略略有不均
5. 少量术语不统一但不影响理解
6. 个别操作步骤表达不够细
7. 个别段落重复表达

Suggestion

代码块

1. 可优化措辞
2. 可增加截图
3. 可补充操作注意事项
4. 可增强章节过渡
5. 可补充更多示例说明
6. 可进一步提升排版观感

11. FrozenDocPlan 的作用

FrozenDocPlan 是用户确认后的“文档生成合同”。

它控制：

代码块

1. 软件身份
2. 诊断快照
3. 模板决策
4. 章节策略
5. 功能策略
6. 证据策略
7. 截图策略
8. 写作策略
9. Quality Gate 策略

关键规则：

代码块

- 1 用户确认模板后，OrchestratorAgent 生成 FrozenDocPlan。
- 2 OutlineAgent 只能基于 FrozenDocPlan 生成目录。
- 3 WriterAgent 只能基于 FrozenDocPlan 写正文。
- 4 AuditAgent 要检查草稿是否违反 FrozenDocPlan。

12. State 设计

当前 State 版本：DocForgeState v2.0

State 用于 LangGraph 全流程状态追踪。

核心原则：

代码块

- 1 State 不保存完整 docx、完整截图、完整原始资料。
- 2 State 保存文件路径、source_id、Qdrant collection 名、检索结果 id、结构化摘要和当前工作流结果。

三类存储分工：

代码块

- 1 原始资料：本地文件系统
- 2 可检索内容：Qdrant
- 3 当前任务进度：State / state.json

13. 最终产物

用户下载产物：

代码块

- 1 《软件著作权文档》.docx

如果 v3 仍未通过：

代码块

- 1 《软件著作权文档_风险版》.docx

系统内部必须保留但不作为用户下载产物：

代码块

```
1 state.json
2 evidence_map
3 audit_report
4 draft_v1 / draft_v2 / draft_v3
```

14. DOCX 导出策略

AI 不直接生成 DOCX。

正确流程：

代码块

```
1 AI 生成结构化章节内容
2   ↓
3 系统用 python-docx 按固定模板写入 DOCX
```

MVP 基础格式：

代码块

```
1 纸张：A4
2 中文字体：宋体
3 英文字体：Times New Roman
4 正文字号：小四
5 一级标题：黑体三号
6 二级标题：黑体四号
7 正文行距：1.5 倍
8 图片宽度：不超过页面宽度
9 图片标题：图 1-1 xxx
```

15. 建议下一轮对话的目标

下一轮新对话专门做：

代码块

建议新对话依次完成：

代码块

- 1 1. 确定项目工程架构
- 2 2. 确定最终技术栈和依赖包
- 3 3. 设计项目目录结构
- 4 4. 设计配置文件方案
- 5 5. 设计 Provider Adapter
- 6 6. 设计 EmbeddingProvider
- 7 7. 设计 EvidenceStore / QdrantStore
- 8 8. 设计 LangGraph 节点
- 9 9. 划分 Sprint
- 10 10. 为每个 Sprint 写 Cursor 提示词
- 11 11. 逐步让 Cursor 落地代码

16. 下一轮对话建议开场提示词

可以在新对话中直接复制以下内容：

代码块

- 1 我们现在要继续推进“墨衡 DocForge”项目。
- 2
- 3 上一轮已经完成产品设计文档审计，最终结论是：B，设计文档已经具备工程落地的基本要求，可以开始用 Cursor 落地工程。
- 4
- 5 项目定位：
- 6 墨衡 DocForge 是一个面向软件著作权文档生成的多智能体文档生产系统。它根据外部参考软著样例、自家公司产品资料、需求文档，自动完成产品理解、文档规划、初稿生成、审计评分、多轮修订和最终导出。
- 7
- 8 MVP 范围：
- 9 1. 读取参考软著样例，支持 docx / pdf / md / txt；
- 10 2. 读取目标产品资料；
- 11 3. 自动生成软著草稿；
- 12 4. 最多 3 轮审计-修订；
- 13 5. Streamlit 本地轻量前端；
- 14 6. 最终只导出《软件著作权文档》.docx 或《软件著作权文档_风险版》.docx。
- 15
- 16 MVP 不做：
- 17 1. 不接代码仓库；

18 2. 不通过 URL 读取网页;
19 3. 不做复杂权限;
20 4. 不接公司知识库;
21 5. 不做多格式高级排版;
22 6. 不做多租户;
23 7. 不做在线协作编辑;
24 8. 不导出独立审计报告;
25 9. 不导出 PDF;
26 10. 不导出 Markdown。
27
28 已确定技术栈:
29 - Python 3.11 或 3.12
30 - LangGraph
31 - Streamlit
32 - Qdrant Local Persistent Mode
33 - Jina-embeddings-v3
34 - QwenProvider / DeepSeekProvider
35 - python-docx
36 - pypdf / pymupdf
37 - markdown / txt 解析
38 - 图片使用视觉模型解析 + OCR 兜底
39
40 核心工程原则:
41 1. reference_style 资料只能学写法, 不能作为产品事实;
42 2. product_evidence 才能作为产品事实;
43 3. WriterAgent 查询事实时必须通过 Qdrant filter 限定 corpus_type =
product_evidence, allowed_usage = factual_evidence;
44 4. ReferenceStyleAgent 查询风格时必须限定 corpus_type = reference_style,
allowed_usage = style_only;
45 5. 用户确认模板后生成 FrozenDocPlan;
46 6. 后续 OutlineAgent / WriterAgent / AuditAgent 必须服从 FrozenDocPlan;
47 7. AuditAgent 内部生成 audit_report, 但不作为用户下载产物;
48 8. 用户最终只下载 docx 文件。
49
50 本轮目标:
51 请你作为我的工程落地顾问, 帮我把这个设计文档拆成可以给 Cursor 开发的落地方案, 包括:
52 1. 项目工程架构;
53 2. 目录结构;
54 3. 技术栈依赖;
55 4. 配置文件设计;
56 5. 核心模块拆分;
57 6. LangGraph 节点设计;
58 7. Sprint 划分;
59 8. 每个 Sprint 的验收标准;
60 9. 每个 Sprint 给 Cursor 的提示词。
61
62 要求:

- 63 1. 先不要直接写 Cursor 提示词，先帮我确认工程架构和 Sprint 拆分是否合理；
- 64 2. 等方案稳定后，再逐个 Sprint 输出 Cursor 提示词；
- 65 3. Cursor 提示词必须编号；
- 66 4. Cursor 提示词必须放在 text 代码块中；
- 67 5. 给我的终端命令必须一条命令一个代码块，并说明预期结果；
- 68 6. 我使用 Windows 11 + PowerShell，但本项目设计运行环境是 macOS / Python 3.11 或 3.12，需要注意跨平台路径和命令差异。

17. 下一轮最重要的提醒

下一轮不要一上来让 Cursor 写完整系统。

正确节奏：

代码块

- 1 先架构，再 Sprint，再提示词，再代码。

优先落地顺序建议：

代码块

- 1 Sprint 0: 项目骨架 + 环境配置
- 2 Sprint 1: Streamlit 上传界面 + 本地文件存储
- 3 Sprint 2: SourceItem / EvidenceItem / DocForgeState / FrozenDocPlan 类型定义
- 4 Sprint 3: 文档解析 + Evidence Unit 生成
- 5 Sprint 4: Qdrant Local Persistent 写入与检索
- 6 Sprint 5: Provider Adapter + EmbeddingProvider
- 7 Sprint 6: LangGraph 主流程骨架
- 8 Sprint 7: 核心 Agent 节点实现
- 9 Sprint 8: AuditAgent + Quality Gate
- 10 Sprint 9: DOCX ExportAgent
- 11 Sprint 10: 端到端测试与样例软著生成

这一轮交接到此结束。